

Project 2 – Cryptographic Protocols

Due | 03.06.2026

TU Wien

1 General Remarks

This project focuses on a cryptographic protocol implementing a simplified version of the SSH key exchange and authentication, and requires the use of the Lean proof assistant and the ProVerif protocol verifier to validate some secrecy and authentication properties of the protocol.

Specifically,

Lean You are asked to show that all processes encoding the protocol are well typed. This step ensures that the protocol provides both secrecy and integrity for the secure channel established between the client and the server.

PROVERIF You will then encode the protocol in the input language of PROVERIF, to check for specific authentication, secrecy and confidentiality properties.

2 Setting up Your System

This homework requires verifying your answers using a recent version of Lean4 and the PROVERIF protocol verifier.

- To use Lean4 it is recommended to use the VSCode IDE. Although other editors may support Lean, the official documentation only applies to VSCode.
 - To install VSCode, follow the instructions on the official website.
 - To install the Lean VSCode extension, search for "Lean 4" in the search bar of the *Extensions* sidebar and click on the install button.
 - Access the Lean 4 setup guide: (i) Create a new text file by selecting *File > New Text File*; (ii) click on the $\forall$ symbol at the top-right corner of the window and select *Documentation > Docs: Show Setup Guide* From the dropdown menu; (iii) read and follow the instructions provided to complete the installation.
- PROVERIF can be downloaded from the official website at <https://proverif.inria.fr>. It also available in the AUR, via opam or the nix store.
 - The website also includes an extensive manual, documenting the usage of all the features of the tool.
 - Your submissions will be checked using PROVERIF 2.05.

3 Submission Instructions

Please submit your solution by *03.06.2026* on TUWEL. This project will account for 18 out of the 60 project points.

Your submission must be a single archive file `name-matriculation.zip` containing

1. The `submission.pdf` file containing all your answers and explanations, a $\text{\LaTeX}$ template can be found on TUWEL.
2. The Lean and PROVERIF files containing your solutions, as described in the submission box of each task:

Required files	Optional Files
SSH.lean ssh_auth.pv ssh_full.pv ssh_mitm.pv ssh_fwd.pv	

4 SSH (Secure Shell)

Secure Shell (SSH) is a widely used cryptographic protocol that provides a secure channel for managing remote servers. It was originally designed as a replacement for Telnet to provide secure shell access to remote systems, but has since been extended to support file transfer (SFTP), port forwarding, tunneling, and even X11 forwarding.

The security foundations of SSH are based on authenticated key exchange and symmetric encryption. The protocol establishes a secure, authenticated channel between a client and a server before any data is transmitted. This ensures that communication is protected against eavesdropping, tampering, and man-in-the-middle attacks.

In this project, we analyze a simplified version of the SSH key exchange protocol and its authentication subprotocol. While the full SSH protocol is considerably more complex, this simplified version captures the essential security properties and vulnerabilities that we will investigate using Lean and PROVERIF.

5 A Simplified SSH Protocol

Let us start by analyzing the simplified SSH key exchange shown in Fig. 1. The protocol involves a client and a server establishing a secure, authenticated channel. The server has a long-term signing key pair (sk_S, vk_S) for authentication, and both parties generate fresh exponents a and b for the Diffie-Hellman key exchange. The protocol uses a trusted group generator g in group $\mathcal{G}$ where the Diffie-Hellman problem is assumed to be hard.

In real SSH, the server signs an *exchange hash* computed over the full session transcript (version strings, algorithm negotiation, DH values, and shared secret). In our simplified model, the server instead signs a hash of the Diffie-Hellman public values $h(g^a, g^b)$, which captures the essential property: binding the servers authentication to the specific key exchange.

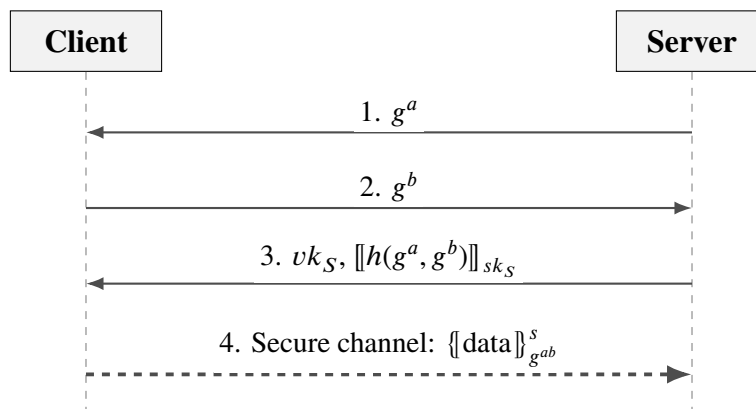


Figure 1: Simplified SSH Key Exchange and Server Authentication Protocol.

We use the notation $\llbracket m \rrbracket_k$ for signatures (signed with key k), and $\llbracket m \rrbracket_k^s$ for symmetric encryption with key k , a, b are fresh exponents, and the secure channel (message 4) uses symmetric encryption with key g^{ab} .

The protocol is composed of two main phases:

Key Exchange Phase (Messages 1-2). Both parties exchange Diffie-Hellman public values g^a and g^b . These are exchanged over the public network and can be observed by an adversary. After this exchange, both parties can compute the shared secret g^{ab} which will be used to encrypt subsequent communications.

Authentication Phase (Message 3). The server authenticates itself by signing the hash of the exchanged Diffie-Hellman public values $h(g^a, g^b)$ with its long-term signing key sk_S and sending both the signature $\llbracket h(g^a, g^b) \rrbracket_{sk_S}$ and the verification key vk_S (message 3). This binds the servers identity to the specific key exchange, preventing man-in-the-middle attacks when the server verification key is known by the client. The client verifies this signature to authenticate the server. Both parties can now use the shared secret g^{ab} for symmetric encryption to establish a secure channel. Client authentication is handled separately by the public key authentication subprotocol (Section 6).

6 SSH Public Key Client Authentication

Once the secure channel has been established through the key exchange, the server must verify the clients identity before granting access. SSH supports several authentication methods; in this project, we focus on public key authentication, shown in Fig. 2. In this sub-protocol, the client proves ownership of a private key sk_C whose corresponding verification key vk_C is registered as an authorized key on the server. All messages are transmitted over the secure channel established during key exchange, using symmetric encryption with the shared key g^{ab} .

As with the key exchange, we simplify the session identifier: in real SSH, sid is a hash over the full key exchange transcript; in our model, we define $sid = h(g^a, g^b)$, the hash of the Diffie-Hellman public values known to both parties after the key exchange.

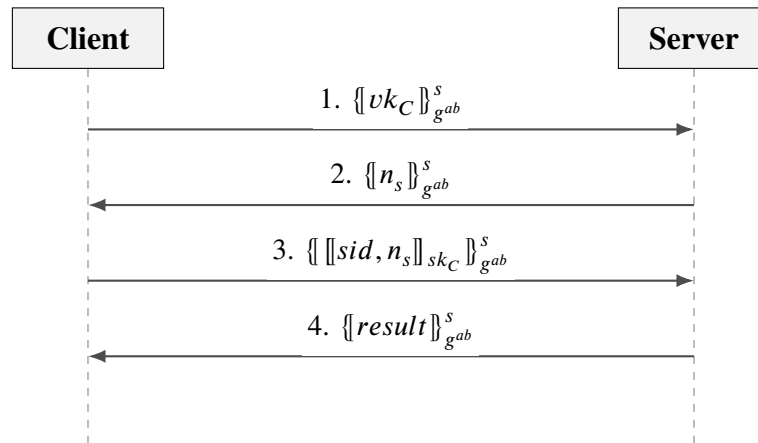


Figure 2: SSH Public Key Client Authentication Protocol.

The protocol consists of two main phases.

Key Query Phase (Message 1). The client initiates authentication by sending its verification key vk_C to the server over the secure channel. The server checks whether vk_C is registered in its set of authorized keys. If the key is not recognized, the server rejects the authentication attempt immediately.

Challenge-Response Phase (Messages 2-4). If the key is authorized, the server generates a fresh nonce n_s and sends it as a challenge (message 2). The client proves ownership of the corresponding private key sk_C by signing both the session identifier sid and the challenge nonce n_s , then sending the signature over the secure channel (message 3). The inclusion of sid in the signature binds the authentication to the current session, preventing cross-session replay attacks. The server verifies the signature using vk_C and sends the authentication result (message 4). In real SSH, this corresponds to an `SSH_MSG_USERAUTH_SUCCESS` or `SSH_MSG_USERAUTH_FAILURE` message; in our model, it is an abstract accept/reject acknowledgment.

7 Type Systems for Secrecy and Integrity

Let us now focus on the public key client authentication subprotocol (Fig. 2). Assuming the Diffie-Hellman key exchange is secure, we have a shared secret g^{ab} that can be used to encrypt data between the client and the server. The authentication protocol uses this symmetric key alongside the clients signing key sk_C .

The authentication subprotocol is well typed, and thus preserves secrecy and integrity of the communicated data, when we assign the following types to the keys used in the protocol.

$$\begin{array}{ll} g^{ab} & \text{SymK}^{HH} \llbracket HH \rrbracket \\ sk_C & \text{SigK}^{HH} \llbracket HH \rrbracket \end{array}$$

Table 1: Types of the keys used in the SSH public key authentication protocol

The shared secret g^{ab} has type $\text{SymK}^{HH} \llbracket HH \rrbracket$: the key has high confidentiality and high integrity (HH), and the data it encrypts also has high confidentiality and high integrity (HH), since messages over the secure channel should remain secret and untampered. The signing key sk_C has type $\text{SigK}^{HH} \llbracket HH \rrbracket$: it operates at high confidentiality and integrity (HH) since the signed content (the session identifier and challenge nonce) is sent through the encrypted channel.

7.1 Project Task: Proving that the Subprotocol is Well-Typed

The `leanspi` subfolder of the course repository contains the implementation of the type system that was presented during the lectures.

- <https://gitlab.secpriv.tuwien.ac.at/teaching/fmsp-s26/-/tree/main/leanspi>

The file `Leanspi/Spi/SSH.lean` is a template containing the definitions of the processes implementing the messages of the public key client authentication subprotocol (Fig. 2). The template includes a partial proof that the full subprotocol is well-typed and the unproven statements of two theorems:

1. (4pts) Prove that the process implementing the client is well typed in the well-formed environment where the network is a LL channel and the keys are typed according to Table 1.
Theorem name: `client_wt`.
2. (4pts) Prove that the process implementing the server is well typed in the well-formed environment where the authorized key of the server has type $\text{VerK}^{LH} \llbracket HH \rrbracket$.
Theorem name: `server_wt`.
3. (1pt) Complete the proof that the composition of the above processes is well typed.
Theorem name: `ssh_wt`.

Your task is to complete the three proofs applying the `Process.WellTyped` and `Term.HasType` typing rules. The template includes the definition of three tactics that can be used to automate some proof obligations.

Submission (9pt). Please add to your submission the modified `SSH.lean` file. We consider a theorem proven if it does not contain the `sorry` keyword (or depends on external axioms).

7.2 Project Task: Discussion

Let us now consider the process encoding the server behavior (`server` in `SSH.lean`).

1. (1pt) In the version of the protocol shown in Fig. 2, the client signs a pair containing both the session identifier and the server nonce, i.e. $\llbracket sid, n'_s \rrbracket_{sk_C}$ whereas in our Lean encoding we only sign the nonce: $\llbracket n'_s \rrbracket_{sk_C}$. This simplification is necessary because our current Lean implementation of the type system does not support tuple/pair types. Suppose we extend the type system with a new ad hoc typing rule allowing multiple values to be assigned a single type:

$$\Gamma \vdash M_1, \dots, M_n : T$$

What should the type T be in relation to the types of M_n , and why? In particular, explain how we would type $\llbracket sid, n'_s \rrbracket_{sk_C}$, assuming $\Gamma \vdash sk_C : \text{SigK}^{HH} \llbracket HH \rrbracket$ and $\Gamma \vdash sid : LH$.

(Hint: our security labels form a lattice that is consistent with the subtyping rule)

2. (1pt) In our type system, verification keys must have low confidentiality. In the server code, the received verification key is compared against the trusted `authorized_key` before we use `authorized_key` to verify the signature. Suppose we remove this equality check and instead verify the signature directly using the received key `vkC`. Would the process still typecheck? Why or why not? Discuss the interaction between the confidentiality level of verification keys and the fact that `vkC` is received from a high-confidentiality channel.

Submission (2pt). Explain/Discuss the above points in section 8.2 of the `submission.pdf` report.

8 Proving Secrecy and Authentication with PROVERIF

Let us now analyze the simplified SSH protocol with PROVERIF. The following suggestions may help you during the process and in preparing your submission:

1. If you encounter any problems, be sure to discuss them in your `submission.pdf` report. This helps us grading non-working PROVERIF scripts. And do not hesitate to ask questions in the forum!
2. Make sure your protocol executes (i.e., that the end of every subprocess is reachable), otherwise PROVERIF queries are meaningless. That is, any secrecy or authentication query is trivially true if the protocol never executes.
3. Public keys and identifiers should be public! Don't forget to make sure the attacker has access to them.
4. Ideally, we want to model a system composed of multiple sessions executed in parallel. For instance, we may want multiple clients and servers, and we want them to have multiple sessions.
 - In this setting, PROVERIF can quickly reach its limits—for example, it may not terminate. If you encounter one of such cases, consider reducing the level of parallelism to find a good middle ground between generality and the ability of PROVERIF to analyze the protocol. Discuss the issue and justify why your chosen compromise is suitable in the relevant section of your `submission.pdf` report.
5. If you feel that additional files are needed to clarify a point, don't hesitate to include them in your submission. Be sure to name them clearly and explain their purpose in your `submission.pdf` report.

8.1 Warmup

Let us focus on the SSH public key client authentication subprotocol shown in Fig. 2 (Fig. 2). Assuming the Diffie-Hellman key exchange is secure, we have a shared secret g^{ab} that can be used to encrypt data between the client and the server. Model only the four authentication messages of this subprotocol in the PROVERIF input language. Include the definitions of the required constructors and destructors. Specify a secrecy query to verify that the fresh server challenge n_s remains secret from the attacker, and a authentication query checking that the subprotocol correctly authenticates the client.

Submission (3pts). Model the authentication subprotocol (Fig. 2) in PROVERIF (with relevant queries, see above). Put your model into `ssh_auth.pv`.

Discuss your modeling choices in Section 8.1 of the `submission.pdf` report (e.g., how you represented the shared secret g^{ab} and the secure channel it provides, and your choices regarding suggestion 4 above).

8.2 The Full Protocol

We now shift our focus to the full SSH protocol, combining both the key exchange and server authentication (Fig. 1) with the client authentication subprotocol (Fig. 2). Encode in PROVERIF the processes modeling all messages of the combined protocol. Include the definitions of the required constructors and destructors, and specify secrecy queries to confirm that both the shared secret g^{ab} and the client's private key sk_C remain confidential. Additionally, specify a secrecy query to confirm that any data sent to the encrypted channel after authentication remains secret.

What are the security requirements you think are relevant for the full version of the protocol? Encode them as

PROVERIF queries and verify if they are valid. In particular, make sure that the protocol is actually executable (for instance by checking that all relevant events are reachable), and add correspondence queries that capture the intended authentication guarantees of the protocol.

Submission (2pts). Model the full protocol (Fig. 1 + Fig. 2) in `ssh_full.pv`. Which queries do you believe are relevant to this protocol? Add them to the `ssh_full.pv` file and discuss them in Section 8.2 of the `submission.pdf` report.

8.2.1 An Attack

In practice, SSH clients warn the user when a server presents a different host key than expected, for instance with a message such as `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`. Suppose the user ignores this warning and the client no longer checks that the received server verification key matches the expected one. Model this variant and show that it enables a man-in-the-middle attack: the attacker can impersonate the server to the client by presenting its own key and completing the key exchange.

Hint: to obtain such a trace in PROVERIF, it is often useful to enrich the attackers initial knowledge with additional free names, e.g., an attacker-controlled long-term signing key and other protocol material needed to behave like a rogue server.

As part of your analysis, check which of the properties you verified for the full protocol still hold in the presence of the man-in-the-middle attack, and which ones break.

Submission (1pt). Model a buggy version of the protocol in which the client does not validate the server's host key in `ssh_mitm.pv`. Find a man-in-the-middle attack on this version and explain it in Section 8.2 of the `submission.pdf` report. Then describe how to fix the attack, and explain the output of the relevant queries after the fix.

8.2.2 Forward Secrecy

Forward secrecy informally means that compromising a long-term key in the future should not reveal past session keys or past confidential application data. Use PROVERIF to investigate whether the SSH protocol modeled in this project satisfies this property. In particular, consider a model in which the servers long-term signing key is revealed only *after* (e.g., using PROVERIF phases) a session has completed, and check whether previously established secrets protected under g^{ab} remain confidential.

Submission (1pt). Model this forward-secrecy experiment in `ssh_fwd.pv`. Discuss in Section 8.2 of the `submission.pdf` report how your model captures forward secrecy, which queries you used, and what their output tells you about the protocol.